Tyr-Crypto Computational Handbook

Volume 5: AES-CTR

A guided calculation book for AES state layout, S-box substitution, ShiftRows, finite-field multiplication, MixColumns, AES-256 key expansion, rounds, counters, and CTR XOR.

Assumed knowledge: Volume 0. Hex, XOR, shifts, and basic arrays are reviewed again where they matter.

Blue	new input, public data, or plain-language explanation
Purple	exact recipe to follow
Green	fully worked calculation
Orange	exercise for the reader
Red	misuse, security, or implementation danger
Teal	checkpoint before continuing

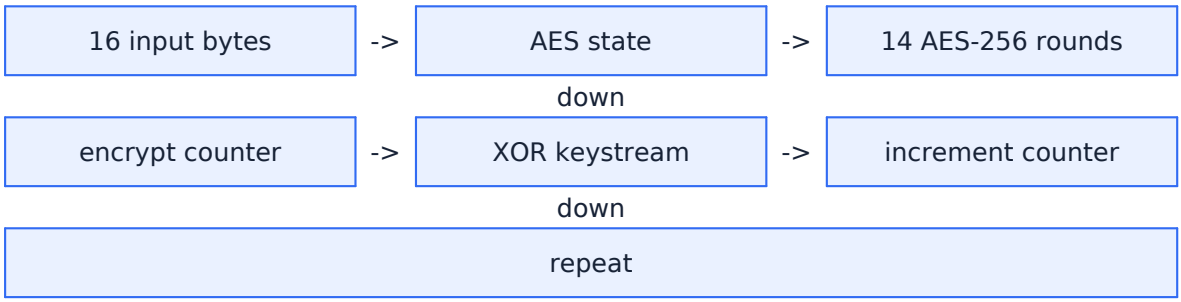
Repository snapshot reviewed: main branch, July 2026.
Square-corner calculation-first teaching edition.

1 What this volume will teach

Read this first

This volume does not ask you to memorize an algorithm diagram. Each page introduces exactly one mechanical action. First read the plain-language box, then copy the rule, then cover the worked examples and reproduce them yourself.

The full path



Study routine for an easily overloaded brain

1. Work for one method only. 2. Say every intermediate value aloud or write it down. 3. Stop at the teal checkpoint. 4. Continue only when the checkpoint feels boring. There is no reward for carrying an unclear step into the next page.

Vocabulary before calculations

Block cipher	A keyed transformation from one fixed-size block to another block of the same size.
State	The current 16 AES bytes arranged as four columns.
S-box	A fixed substitution table that replaces one byte with another.
GF(2^8)	The 256-element byte field used by AES MixColumns arithmetic.
Round key	Sixteen bytes from the expanded key schedule used by one AddRoundKey step.
CTR mode	A mode that encrypts successive counter blocks to create a keystream.
Nonce/counter	The initial 16-byte counter value. It must not repeat under the same key.
Authentication	Evidence that ciphertext was not modified; plain CTR does not provide it.

2 Algorithm overview before the details

The promise of the overview

The boxes below deliberately hide most arithmetic. Their job is only to show where each later calculation belongs. When you meet a calculation page, return here and point to its box.

1	16 input bytes
2	AES state
3	14 AES-256 rounds
4	encrypt counter
5	XOR keystream
6	increment counter
7	repeat

Important scope

The examples often use toy-sized states or selected words. The toy calculation keeps the same order of operations but uses smaller numbers so that a human can finish it. Every toy rule is explicitly marked. Never substitute toy parameters into real cryptographic code.

3 Method 1: place 16 bytes into the AES state matrix

Input:	one 16-byte block
Do:	fill columns from left to right
Output:	a 4x4 byte state

Plain-language explanation

AES prints its state as four rows, but the input byte sequence fills one column at a time. This is the source of many ShiftRows and MixColumns mistakes. Array indices 0,1,2,3 form the first column.

Mechanical rule

State cell at row r, column c stores input byte index r+4c. To recover the byte sequence, read each column top to bottom.

Worked example 1 - bytes 00..0F

Rows become 00 04 08 0C / 01 05 09 0D / 02 06 0A 0E / 03 07 0B 0F.

Worked example 2 - first column

Input AA BB CC DD ... -> column 0 is AA BB CC DD.

Worked example 3 - second column

Indices 4,5,6,7 occupy rows 0,1,2,3 of column 1.

Worked example 4 - locate index 14

14=2+4x3, so input byte 14 lies at row 2, column 3.

Exercises

(a) Place 10 11 ... 1F into four printed rows.
(b) Which input indices form column 2?
(c) What state position contains input index 9?
(d) Read rows [00 04 08 0C]/[01 05 09 0D]/[02 06 0A 0E]/[03 07 0B 0F] back into a byte sequence.

Checkpoint

You can point to index 6 and immediately say row 2, column 1.

4 Method 2: apply AddRoundKey

Input:	16 state bytes and 16 round-key bytes
Do:	XOR matching positions
Output:	a new 16-byte state

Plain-language explanation

AddRoundKey is not decimal addition. Each state byte is XORed with exactly one round-key byte at the same array index. The operation is its own inverse.

Mechanical rule

For i=0..15: state[i] = state[i] XOR roundKey[i]. Work nibble-by-nibble or bit-by-bit; keep two hex digits per byte.

Worked example 1 - single byte

0x57 XOR 0x83 = 0xD4.

Worked example 2 - zeros expose key

0x00 XOR 0xA6 = 0xA6.

Worked example 3 - equal bytes cancel

0xFF XOR 0xFF = 0x00.

Worked example 4 - four-byte word

[00 11 22 33] XOR [0F 0F F0 F0] = [0F 1E D2 C3].

Exercises

(a) 57 XOR 13.
(b) AA XOR 55.
(c) [10 20 30 40] XOR [01 02 04 08].
(d) Why does XORing the same round key again restore the prior state?

Checkpoint

You never carry between bits during AddRoundKey.

5 Method 3: replace bytes with the AES S-box

Input:	one state byte
Do:	use its high nibble as row and low nibble as column
Output:	one substituted byte

Plain-language explanation

The S-box is a fixed 16x16 table. A byte such as 0x53 has row 5 and column 3. Tyr-Crypto’s default path scans all 256 entries with masks rather than indexing the secret byte directly, but the mathematical result is the same table value.

Mechanical rule

Split byte xy into row x and column y. Read SBOX[x,y]. Keep the result as exactly one byte.

Worked example 1 - 0x00

Row 0, column 0 -> 0x63.

Worked example 2 - 0x53

Row 5, column 3 -> 0xED.

Worked example 3 - 0x7C

Row 7, column C -> 0x10.

Worked example 4 - 0xFF

Row F, column F -> 0x16.

Exercises

(a) SBOX(0x01).
(b) SBOX(0x10).
(c) SBOX(0xA5).
(d) Substitute bytes 00 53 7C FF.

Checkpoint

You use the two hex digits as coordinates, not as decimal 53 or decimal 7C.

6 Method 4: shift the four AES rows

Input:	a 4x4 state after SubBytes
Do:	rotate row r left by r positions
Output:	a permuted state

Plain-language explanation

ShiftRows moves bytes but does not alter their values. Row 0 stays still. Row 1 shifts one place, row 2 shifts two, and row 3 shifts three. The movement wraps around within the row.

Mechanical rule

Row0: [a b c d]. Row1: [e f g h] -> [f g h e]. Row2 -> [k l i j] for [i j k l]. Row3 -> [p m n o] for [m n o p].

Worked example 1 - row 0

00 04 08 0C -> 00 04 08 0C.

Worked example 2 - row 1

01 05 09 0D -> 05 09 0D 01.

Worked example 3 - row 2

02 06 0A 0E -> 0A 0E 02 06.

Worked example 4 - row 3

03 07 0B 0F -> 0F 03 07 0B.

Exercises

(a) Shift row1 AA BB CC DD.
(b) Shift row2 10 20 30 40.
(c) Shift row3 01 02 03 04.
(d) Which row is unchanged?

Checkpoint

You can perform ShiftRows without touching bytes in different rows.

7 Method 5: multiply one AES byte by 2 with xtime

Input:	one byte x
Do:	left shift and conditionally XOR 0x1B
Output:	2x in GF(2^8)

Plain-language explanation

AES byte multiplication is polynomial arithmetic, not ordinary integer multiplication. Multiplying by x corresponds to a left shift. If the old top bit was 1, the ninth bit must be reduced using the AES polynomial; in byte form this means XOR 0x1B.

Mechanical rule

shifted=(x<<1) mod 256. carry=top bit of old x. If carry=0, result=shifted. If carry=1, result=shifted XOR 0x1B.

Worked example 1 - no carry: 0x57

0x57<<1=0xAE; top bit was 0 -> 0xAE.

Worked example 2 - carry: 0x83

low byte of 0x106 is 0x06; top bit was 1 -> 0x06 XOR 0x1B = 0x1D.

Worked example 3 - zero

0x00 -> 0x00.

Worked example 4 - top bit only

0x80 -> shifted low byte 0x00; XOR 0x1B -> 0x1B.

Exercises

(a) xtime(0x01).
(b) xtime(0x7F).
(c) xtime(0xC2).
(d) What decides whether 0x1B is XORed?

Checkpoint

Before shifting, you mark the old most significant bit.

8 Method 6: multiply one AES byte by 3

Input:	one byte x
Do:	compute xtime(x), then XOR x
Output:	3x in GF(2^8)

Plain-language explanation

In this field, 3 means the polynomial x+1. Therefore multiplication by 3 is “multiplication by 2” XOR the original byte. It is not xtime(x)+x with carries.

Mechanical rule

mul3(x)=xtime(x) XOR x.

Worked example 1 - 0x57

xtime=AE; AE XOR 57 = F9.

Worked example 2 - 0x83

xtime=1D; 1D XOR 83 = 9E.

Worked example 3 - 0x00

00 XOR 00 = 00.

Worked example 4 - 0x80

xtime=1B; 1B XOR 80 = 9B.

Exercises

(a) mul3(0x01).
(b) mul3(0x7F).
(c) mul3(0xC2).
(d) Explain why ordinary decimal 3x is wrong here.

Checkpoint

You always calculate xtime first, then XOR the unchanged original byte.

9 Method 7: mix one AES column

Input:	four bytes in one state column
Do:	form four XOR sums using coefficients 2,3,1,1
Output:	four mixed bytes

Plain-language explanation

MixColumns lets every output byte depend on every input byte in its column. It never combines different columns. Calculate all 2x and 3x values before assembling the four XOR equations; this prevents repeated work and copy errors.

Mechanical rule

For [a0,a1,a2,a3]: b0=2a0 XOR 3a1 XOR a2 XOR a3; b1=a0 XOR 2a1 XOR 3a2 XOR a3; b2=a0 XOR a1 XOR 2a2 XOR 3a3; b3=3a0 XOR a1 XOR a2 XOR 2a3.

Worked example 1 - DB 13 53 45

Input column [DB 13 53 45]. Compute 2x and 3x terms, then XOR each row equation. Output [8E 4D A1 BC].

Worked example 2 - F2 0A 22 5C

Input column [F2 0A 22 5C]. Compute 2x and 3x terms, then XOR each row equation. Output [9F DC 58 9D].

Worked example 3 - 01 01 01 01

Input column [01 01 01 01]. Compute 2x and 3x terms, then XOR each row equation. Output [01 01 01 01].

Worked example 4 - 80 00 00 00

Input column [80 00 00 00]. Compute 2x and 3x terms, then XOR each row equation. Output [1B 80 80 9B].

Exercises

(a) Mix column 00 00 00 00.
(b) Mix column 01 00 00 00.
(c) Mix column 00 01 00 00.
(d) Why can four columns be calculated independently?

Checkpoint

You can write the four equations from memory and label each intermediate 2x and 3x.

10 Method 8: expand the AES-256 key into round keys

Input:	a 32-byte key and earlier 4-byte words
Do:	apply RotWord/SubWord/Rcon at scheduled positions, then XOR an older word
Output:	60 words for 15 round keys

Plain-language explanation

AES-256 does not reuse the original key unchanged in every round. It expands eight initial words into sixty words. Most new words are a simple XOR with the word eight positions earlier. At every eighth word, the previous word is rotated, substituted, and given Rcon. Halfway through that cycle, SubWord is applied without rotation or Rcon.

Mechanical rule

For i>=8: temp=w[i-1]. If i mod 8=0: temp=SubWord(RotWord(temp)); temp[0] XOR=Rcon. If i mod 8=4: temp=SubWord(temp). Then w[i]=w[i-8] XOR temp.

Worked example 1 - standard w8

w7=1C 1D 1E 1F; Rot/Sub/Rcon then XOR w0 gives w8=A5 73 C2 9F.

Worked example 2 - ordinary w9

w9=w1 XOR w8 -> A1 76 C4 98.

Worked example 3 - special w12

i=12 has i mod 8=4, so SubWord(w11) then XOR w4 -> 16 51 A8 CD.

Worked example 4 - next Rcon point w16

i=16 uses RotWord/SubWord/Rcon again -> AE 87 DF F0.

Exercises

(a) For i=10, is RotWord used?
(b) For i=12, is Rcon used?
(c) How many initial words come from a 32-byte key?
(d) How many 16-byte round keys does AES-256 need?

Checkpoint

You can decide which branch applies from i mod 8 before touching any bytes.

11 Method 9: assemble an AES encryption round

Input:	state and the next round key
Do:	apply transformations in the correct order
Output:	the next state

Plain-language explanation

AES encryption begins with AddRoundKey. Normal rounds then use SubBytes, ShiftRows, MixColumns, AddRoundKey. The final round deliberately omits MixColumns. Tyr-Crypto’s AES-256 core performs fourteen rounds.

Mechanical rule

Initial: AddRoundKey(K0). Rounds 1..13: SubBytes -> ShiftRows -> MixColumns -> AddRoundKey(Kr). Round 14: SubBytes -> ShiftRows -> AddRoundKey(K14).

Worked example 1 - initial step

Plaintext state XOR round key 0. No S-box or MixColumns yet.

Worked example 2 - round 1

SubBytes, ShiftRows, MixColumns, then XOR round key 1.

Worked example 3 - round 13

Still a normal AES-256 round, so MixColumns is included.

Worked example 4 - round 14

Final round: SubBytes, ShiftRows, AddRoundKey. No MixColumns.

Exercises

(a) What is the first operation before round 1?
(b) Does AES-256 round 10 include MixColumns?
(c) Does round 14 include MixColumns?
(d) How many SubBytes stages occur in AES-256 encryption?

Checkpoint

You can write the full operation order without confusing the initial key addition with a normal round.

12 Method 10: increment the 128-bit CTR counter

Input:	one 16-byte counter block
Do:	add one from the rightmost byte with carry to the left
Output:	the next counter block

Plain-language explanation

Tyr-Crypto treats the entire 16-byte nonce parameter as a big-endian 128-bit counter. The rightmost byte is the least significant. Increment it first. Only continue left when a byte wraps from FF to 00.

Mechanical rule

Set carry=1 and start at index 15. value=byte+carry; store low 8 bits; carry=1 only if value exceeded FF. Stop when carry=0.

Worked example 1 - ordinary increment

... 00 09 -> ... 00 0A.

Worked example 2 - one-byte carry

... 12 FF -> ... 13 00.

Worked example 3 - two-byte carry

... 7F FF FF -> ... 80 00 00.

Worked example 4 - full wrap

FF repeated 16 times -> 00 repeated 16 times. This wrap must not be allowed in a secure stream.

Exercises

(a) Increment 00...00.
(b) Increment 00...01 FF.
(c) Increment 00...FF FF FF.
(d) Which end of the array changes first?

Checkpoint

You can increment a printed counter without reversing the byte order.

13 Method 11: encrypt and decrypt in CTR mode

Input:	counter blocks, AES key, and plaintext bytes
Do:	encrypt counters to keystream and XOR with data
Output:	ciphertext of the same length

Plain-language explanation

CTR turns a block cipher into a stream cipher. AES encrypts the counter, not the plaintext. The resulting 16-byte block is XORed with plaintext. Decryption repeats the same operation because XOR is its own inverse.

Mechanical rule

For block j: KSj=AES_K(counter+j). Cj=Pj XOR KSj. Decrypt Pj=Cj XOR KSj. A partial final block uses only the required prefix of KSj.

Worked example 1 - single byte

P=4D, KS=A6 -> C=EB.

Worked example 2 - decrypt same byte

C=EB, KS=A6 -> P=4D.

Worked example 3 - four bytes

P=10 20 30 40, KS=01 02 04 08 -> C=11 22 34 48.

Worked example 4 - partial block

A 20-byte message uses one full 16-byte keystream block and the first 4 bytes of the next.

Exercises

(a) P=00 FF, KS=AA 55.
(b) Decrypt C=AA AA with KS=0F F0.
(c) How many AES counter encryptions for 33 bytes?
(d) Does CTR decryption invoke the AES inverse cipher?

Danger to remember

Reusing the same key and initial counter repeats the keystream. XORing two ciphertexts then cancels the keystream and exposes the XOR of the plaintexts. CTR also provides no authentication by itself.

Checkpoint

You can draw counter -> AES -> keystream -> XOR data, and you never feed plaintext into AES itself.

14 Cumulative guided calculations

How cumulative work differs

Earlier exercises isolated one action. These tasks chain several actions together. Draw a small checkbox after every line. Do not calculate two lines in your head at once.

Cumulative task 1 - One normal AES round

Given a state and round key, list every transformation and identify which ones change values versus positions.

Cumulative task 2 - Counter carry chain

Start with 00...12 FF FE. Write the next three counters.

Cumulative task 3 - CTR partial blocks

A 37-byte message starts at counter N. List counter blocks and used keystream bytes.

Cumulative task 4 - Nonce-reuse failure

C1=P1 XOR K and C2=P2 XOR K. Simplify C1 XOR C2.

15 Tyr-Crypto code map

How to read the code map

The left column names the calculation from this volume. The right column names the current Nim location or implementation behavior. This is a map, not a claim that the implementation is independently audited.

AES constants and contexts	src/protocols/custom_crypto/symmetric/aes/aes_core.nim
Constant-time S-box path	aes_core.nim: sboxCt, subByte; unsafeFastAes enables table lookup
ShiftRows and MixColumns	aes_core.nim: shiftRows, xtime, mul2, mul3, mixColumns
AES-256 key expansion	aes_core.nim: expandKey256
AES-256 encryption rounds	aes_core.nim: encryptBlock(Aes256Ctx, ...)
CTR counter and XOR	src/protocols/custom_crypto/symmetric/aes/aes_ctr.nim
SIMD XOR backends	aes_ctr.nim: SSE2/NEON 16-byte and AVX2 32-byte paths

Security and implementation note

The default AES S-box path scans all 256 entries with equality masks to avoid secret-indexed table access. The compile flag unsafeFastAes intentionally enables a faster table lookup and should be treated as a side-channel tradeoff, not a free optimization.

Security and implementation note

The CTR helper accepts a 32-byte AES-256 key and a 16-byte starting counter. It increments the counter as a big-endian 128-bit integer. The implementation does not add a tag; callers needing integrity must use an authenticated construction and enforce nonce uniqueness.

16 Compact solutions with paths

Use solutions correctly

First compare the final answer. If it differs, compare one intermediate value at a time from left to right. Do not copy the whole line: locate the first point where your work diverged.

Method 1: place 16 bytes into the AES state matrix

Solution path

(a) Rows: 10 14 18 1C / 11 15 19 1D / 12 16 1A 1E / 13 17 1B 1F.
(b) 8,9,10,11.
(c) Row 1, column 2.
(d) 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F.

Method 2: apply AddRoundKey

Solution path

(a) 0x44.
(b) 0xFF.
(c) 11 22 34 48.
(d) Because (x XOR k) XOR k = x; each key bit is toggled twice.

Method 3: replace bytes with the AES S-box

Solution path

(a) 0x7C.
(b) 0xCA.
(c) 0x06.
(d) 63 ED 10 16.

Method 4: shift the four AES rows

Solution path

(a) BB CC DD AA.
(b) 30 40 10 20.
(c) 04 01 02 03.
(d) Row 0.

Method 5: multiply one AES byte by 2 with xtime

Solution path

(a) 0x02.
(b) 0xFE.
(c) 0x9F.
(d) The old top bit (bit 7).

Method 6: multiply one AES byte by 3

Solution path

(a) 0x03.
(b) 0x81.
(c) 0x5D.
(d) AES uses polynomial addition, which is XOR without carries, not integer addition.

Method 7: mix one AES column

Solution path

(a) 00 00 00 00.
(b) 02 01 01 03.
(c) 03 02 01 01.
(d) The MixColumns matrix is applied separately to each 4-byte column.

Method 8: expand the AES-256 key into round keys

Solution path

(a) No; i mod 8=2, so w10=w2 XOR w9.
(b) No; i mod 8=4 uses only SubWord.
(c) 8 words.
(d) 15 round keys: initial key addition plus 14 rounds.

Method 9: assemble an AES encryption round

Solution path

(a) AddRoundKey with K0.
(b) Yes.
(c) No.
(d) 14: once in every numbered round.

Method 10: increment the 128-bit CTR counter

Solution path

(a) 00…01.
(b) 00…02 00.
(c) 00…01 00 00 00.
(d) The rightmost byte, index 15.

Method 11: encrypt and decrypt in CTR mode

Solution path

(a) AA AA.
(b) A5 5A.
(c) 3 counter blocks.
(d) No. Both directions use the AES forward encryption of counter blocks.

17 Cumulative solutions

Cumulative solution 1 - One normal AES round

SubBytes changes each byte value. ShiftRows moves byte positions. MixColumns changes values within each column. AddRoundKey XORs values with matching key bytes.

Cumulative solution 2 - Counter carry chain

00...12 FF FF; 00...13 00 00; 00...13 00 01.

Cumulative solution 3 - CTR partial blocks

Encrypt N for bytes 0..15, N+1 for 16..31, and N+2 for bytes 32..36; only five bytes of the third keystream block are used.

Cumulative solution 4 - Nonce-reuse failure

C1 XOR C2=(P1 XOR K) XOR (P2 XOR K)=P1 XOR P2 because K XOR K=0. This is why repeating a CTR keystream is dangerous.

18 Primary references

Tyr-Crypto AES core	https://github.com/siriuslee69/Tyr-Crypto/blob/main/src/protocols/custom_crypto/symmetric/aes/aes_core.nim
Tyr-Crypto AES-CTR	https://github.com/siriuslee69/Tyr-Crypto/blob/main/src/protocols/custom_crypto/symmetric/aes/aes_ctr.nim
NIST FIPS 197 updated AES standard	https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf
NIST SP 800-38A block-cipher modes	https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf

Safety boundary

This handbook explains calculations and implementation structure. It does not certify Tyr-Crypto or any custom cryptographic implementation for production use. Protocol composition, randomness, nonce management, compiler behavior, side channels, key erasure, and independent review remain separate requirements.